

EE5176: Computational Photography
Programming Assignment-2: Motion Deblurring Report

Ritabrata Mandal
EE25S009

October 1, 2025

0. Introduction

This assignment explores imaging scenarios that result in motion blur and examines various methods to reduce it.

(a) Problem Description

Motion blur can occur when the camera moves while capturing a static scene, when an object in the scene is moving, or when both effects are present. Even with knowledge of the blur kernel, deblurring remains a challenging task in conventional photography. Assuming a 1D blur with a box-shaped kernel, its frequency-domain representation is a sinc function. A straightforward approach to deblurring is to divide the frequency response of the blurred image by that of the transfer function, and then estimate the deblurred image using the inverse transform. However, this method suffers from noise amplification, especially when the transfer function approaches zero. An additional complication arises with non-uniform blur, which often occurs when multiple objects move within the scene along with camera motion. Estimating separate blur kernels for each object requires segmentation, a difficult and error-prone task. For this reason, uniform blurring across the image is desirable, as it avoids the need for segmentation.

(b) Implementation

The implementation has been carried out and tested in python 3.12.10. All processed images generated during this assignment are available in the GitHub repository under the `output` folder.

(c) Running the Code

To execute the assignment, navigate to the `Assignment2` folder, run the `main.ipynb` file.

1. Motion Deblurring with Conventional Camera

We are given a clean image of a scene having M and N columns (`fist.png`) shown in Figure 1. We capture the scene using a static conventional camera with the aperture kept open for the full exposure time. The exposure time is 52 seconds. For simplicity, consider that the camera captures the scene at times $t = 0, \dots, 51$. At time $t = 0$, the image has zero translation. Then, the whole image as seen by the camera moves at 1 pixel per second to the right (horizontal translation).

Original Image



Figure 1: Original Image

(a) **Generating the Blurred Image**

We generate the blurred image having M and $N + 1$ columns captured by the camera. We generate the translated versions of the image for $t = \{0, 1, \dots, 51\}$ and average them. We add Gaussian noise mean 0 and standard deviation 1 (with respect to the maximum intensity of 255) from `gaussNoise.mat`. The blurred image is shown in Figure 2.

Blurred (Conventional)



Figure 2: Blurred image with conventional camera

(b) **Blur Matrix**

We form the blur matrix A of size $(N + 51) \times N$ corresponding to the

horizontal translation described above. This matrix will be a Toeplitz matrix. Figure 3 shows the matrix A .



Figure 3: Toeplitz matrix(conventional)

(c) **Least Squares Deblurring & RMSE**

We deblur the noisy image using the least squares and the generated blur matrix A . We perform the operation on each channel of the image. If the blurred image channel given by y , the reconstructed image channel estimate $\hat{x}$ is given by

$$\hat{x} = yA(A^T A)^{-1}$$

Where A is blur(toeplitz) matrix. Figure 4 shows the resulting deblurred image. The RMSE between the original image and deblurred image is 0.33359.

Blurred (Code Sequence 1)



(a) Blurred(Code Sequence 1)

Blurred (Code Sequence 2)



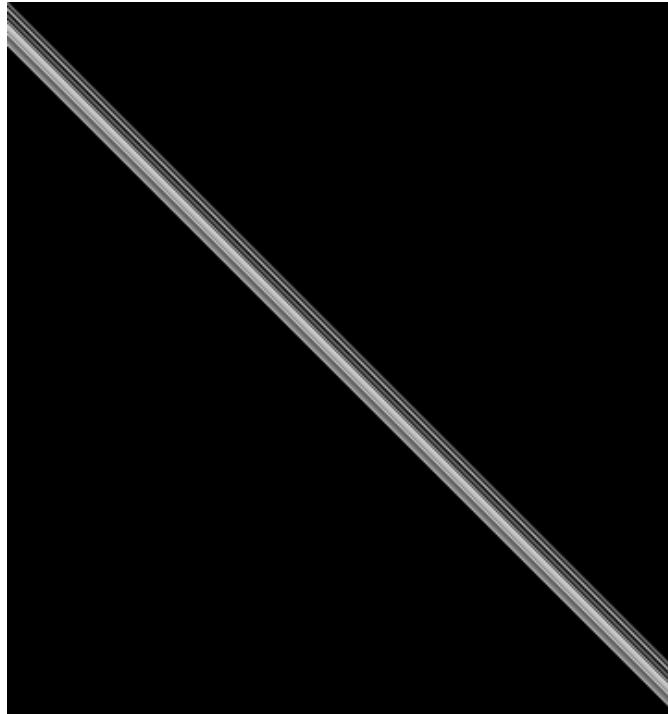
(b) Blurred(Code Sequence 2)

Figure 5: Blurred image with flutter shutter camera

(b) **Blur Matrix**

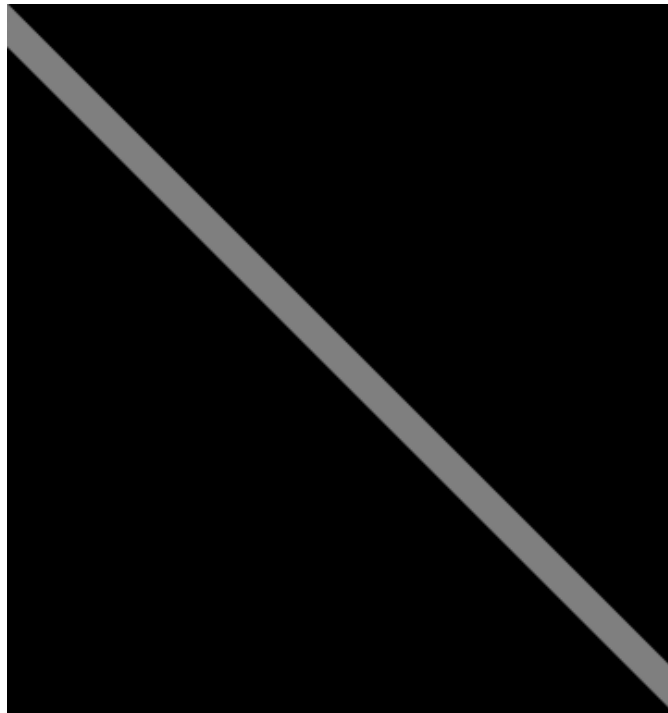
We construct the blur matrices A in the same manner as in the conventional camera case. Figure 6a illustrates the matrix A for code sequence 1, while Figure 6b shows the corresponding matrix for code sequence 2.

Toeplitz Matrix (Code Sequence 1)



(a) Blur matrix(Code Sequence 1)

Toeplitz Matrix (Code Sequence 2)



(b) Blur matrix(Code Sequence 2)

Figure 6: Blurred matrices with different code sequence

(c) **Frequency Response**

We compare the frequency responses of the DFTs corresponding to the conventional and flutter shutter codes, using the first column of the respective A matrices. The magnitude in decibels is plotted in Figure 7. The frequency response of the flutter shutter is clearly more desirable, as it does not exhibit dips caused by zeros and does not show a significant reduction in magnitude at higher frequencies. This implies that images obtained using the flutter shutter are easier to deblur compared to those from the conventional case. Moreover, due to the absence of zeros, we do not expect the noise in the deblurred image to be amplified as it is in the conventional camera.

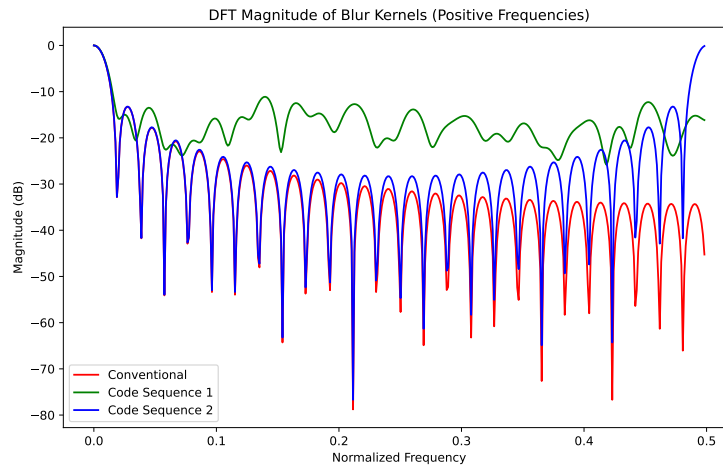


Figure 7: DFT Magnitude of Blur Kernels

(d) **Least Squares Deblurring**

Similar to the conventional case, we estimate the deblurred image from the A matrices. Figure 8a and Figure 8b show the reconstructed images obtained using the least squares method.

Reconstructed (Code Sequence 1)



(a) Reconstructed(Code Sequence 1)

Reconstructed (Code Sequence 2)



(b) Reconstructed(Code Sequence 2)

Figure 8: Reconstructed images

(e) **RMSE & result comparison**

For the reconstructed image using code sequence 1, we observe less noise compared to code sequence 2. This is because the DFT of the first sequence is more broadband, leading to reduced amplification of the Gaussian noise added to the blurred image. The RMSE values also support this observation: 0.033 for code sequence 1 versus 0.2033 for code sequence 2.

(f) **Reconstruction without noise added**

In this case, we don't add Gaussian noise to the image during deblurring

and perform the same deblurring procedure as before. Figure 9a, Figure 9b, and Figure 9c show the blurred images obtained with the conventional camera, flutter shutter code sequence 1, and code sequence 2, respectively. Similarly, Figure 9d, Figure 9e, and Figure 9f present the reconstructed images for the conventional camera, flutter shutter with code sequence 1, and code sequence 2, respectively. Since the blurred images do not contain noise in this case, the differences between the reconstructed images using the two code sequences are very subtle.

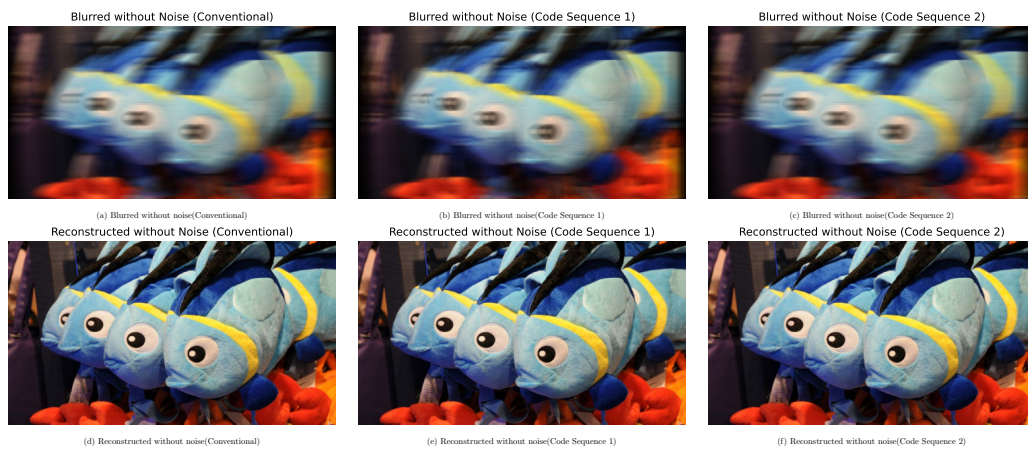


Figure 9: Blurred and reconstructed images without noise

3. Deblurring with motion-invariant photography

We are given images of the top view of a road scene `background.png` and a foreground moving object `redcar.png`.

(a) Static camera

We assume a static camera and model the car's velocity as 1 pixels per second to the right (horizontal translation) relative to the camera. A blurred image is generated for an exposure time of 52 seconds. To simulate this, we first create individual frames for each second, resulting in 52 frames in total. For each frame, we apply the corresponding translation to the foreground object and then merge it with the background. During merging, all pixels with non-zero values in the foreground image are assigned the corresponding foreground pixel values, otherwise, they are assigned background pixel values. Finally, we average the 52 frames to obtain the blurred image. The resulting images are shown in Figures 10.

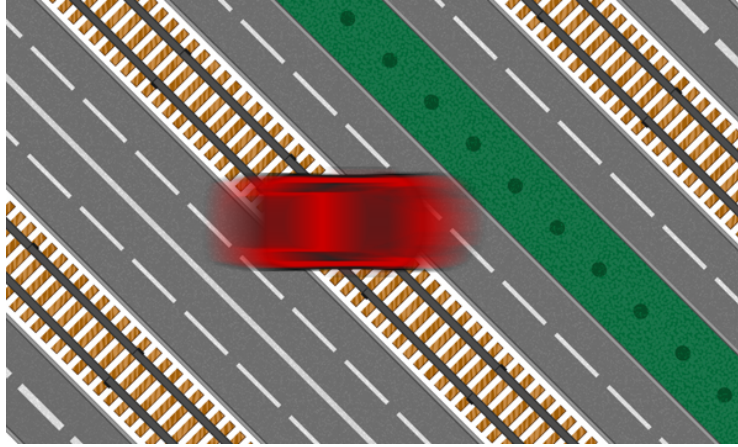


Figure 10: Total image

(b) **Motion-Invariant-Photography**

We now consider the same scene captured using motion-invariant photography, where the camera itself moves during the exposure. The translational values of the static background, caused by camera motion at each second, are provided in `CameraT.mat`. In this case, we generate the blurred image with both camera and object motion for an exposure time of 53 seconds (note that `CameraT` contains 53 entries, not 52).

The process is as follows:

- For each second, we apply the camera-induced translations to the background image and the relative translations to the moving object independently, and then merge them to form a single frame. The relative motion of the object with respect to the moving camera is calculated as $\text{relative translation} = \text{static background translation due to camera} - \text{object translation}$
- Since the background translations caused by the camera are non-integer values, bilinear interpolation and target-to-source mapping are used to generate the translated frames.
- Finally, we average all 52 frames to obtain the motion-invariant blurred image.

The resulting output is shown in Figure 11.

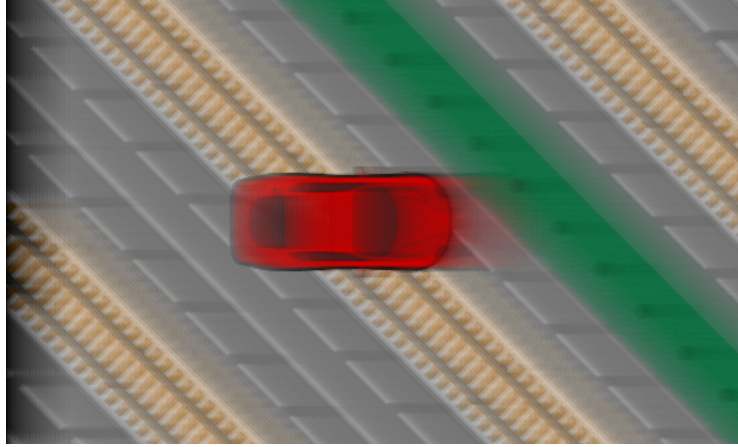


Figure 11: Total image

(c) **Comparison**

In case (a), the foreground appears blurred while the background remains sharp, as both the background and the camera are stationary. In contrast, in case (b), both the foreground and background appear blurred because the camera is in motion, and the relative velocity between the camera and the foreground is non-zero.

(d) **PSF for Motion-Invariant Photography**

We plot the PSF of the blurred image for the case of motion-invariant photography. If x is the camera translation, the PSF weight at x is $w(x) = 1/x^2$ where $x = \{0.1, 1, 2 \cdots 52\}$. We use 52 instead of 51 as given in the assignment to ensure that the shape of the blur matrix generated using it is valid for the blurred image. Figure 12 shows the normalized PSF

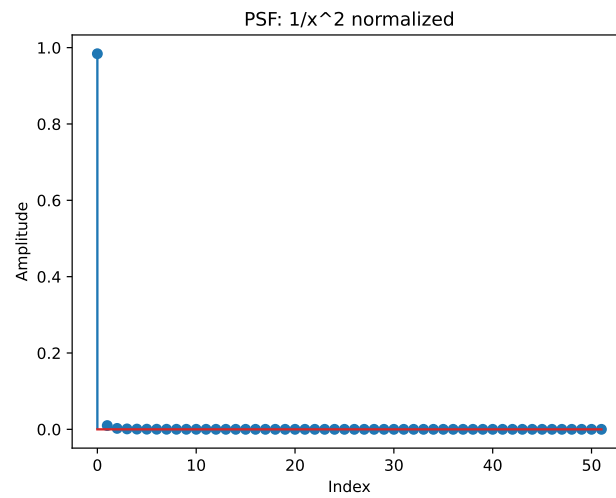


Figure 12: Normalized PSF for motion invariant photography

(e) **Blur Matrix**

We construct the blur matrices A in the using the given PSF. Figure 13

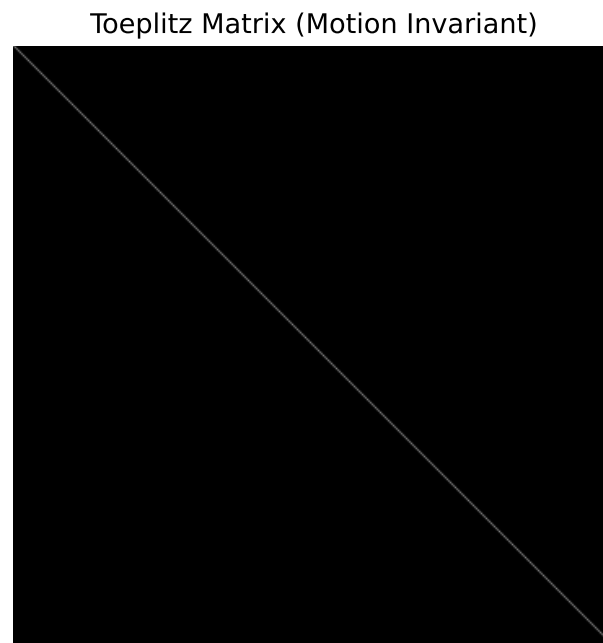


Figure 13: Blur matrix(Motion Invariant)

(f) **Deblurred image**

Similar to the conventional case, we estimate the deblurred image of Figure 11 using A matrix. Figure 14 show the reconstructed images obtained using the least squares method.

Reconstructed Image (Motion Invariant)

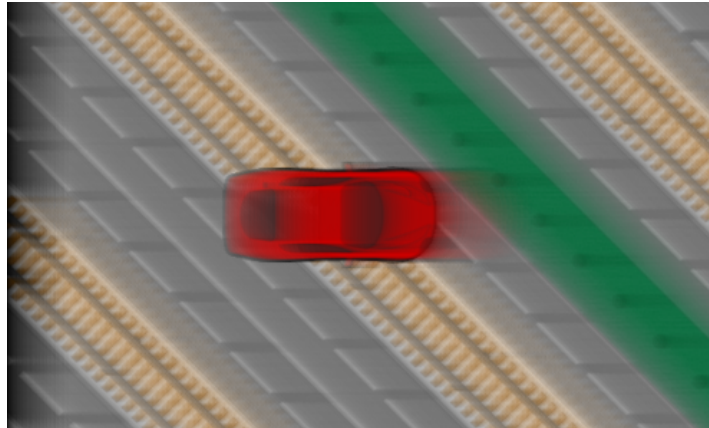


Figure 14: Reconstructed(motion invariant)